DVA Contest 2

100 Practice MCQs — Mid to Hard Level

NumPy · Pandas · Data Cleaning · Filtering · Grouping
Merging · Pivoting · Statistics · Correlation · Outliers · Window Functions

Instructions

- Each question has exactly **one** correct answer.
- Questions marked **[HARD]** require deeper reasoning.
- Code-based questions — trace the output carefully.
- Answer key is on the **last page**.

0 1

1 NumPy — Arrays, Indexing & Slicing 2

2 NumPy — Boolean Masking & Operations 3

3 Pandas — Data Loading & Inspection 4

4 Data Cleaning 5

5 Data Filtering 6

6 Data Grouping 8

7 Data Merging 9

8 Data Pivoting 10

9 Descriptive Statistics 11

10 Frequency Analysis 12

11 Correlation Analysis 13

12 Outlier Detection 14

13 Window Functions 15

14 Mixed / Tricky Code Questions 16

1 1

Q1 | Array Shape

What is the output of the following?

```
import numpy as np
a = np.array([[1,2,3],[4,5,6]])
print(a.shape)
```

(A) (6,) **(B)** (2,3) **(C)** (3,2) **(D)** (2,3,1)

Q2 | Array Slicing

Given `a = np.arange(10)`, what does `a[2:8:2]` return?

(A) `[2, 4, 6]` **(B)** `[2, 4, 6, 8]` **(C)** `[3, 5, 7]` **(D)** `[2, 3, 4, 5, 6, 7]`

Q3 | 2D Slicing

```
b = np.array([[10,20,30],[40,50,60],[70,80,90]])
print(b[1:, ::2])
```

(A) `[[40,60],[70,90]]` **(B)** `[[40,50,60],[70,80,90]]` **(C)** `[[40,60]]` **(D)** `[[50,60],[80,90]]`

Q4 | Reshape

Which of the following will raise a **ValueError**?

(A) `np.arange(12).reshape(3,4)` **(B)** `np.arange(12).reshape(2,6)`
(C) `np.arange(12).reshape(4,4)` **(D)** `np.arange(12).reshape(2,-1)`

Q5 | Negative Indexing

```
a = np.array([5, 10, 15, 20, 25])
print(a[-3:-1])
```

(A) `[15, 20]` **(B)** `[15, 20, 25]` **(C)** `[10, 15]` **(D)** `[20, 25]`

Q6 | [HARD] View vs Copy

```
a = np.array([1, 2, 3, 4, 5])
b = a[1:4]
b[0] = 99
print(a)
```

(A) `[1, 2, 3, 4, 5]` **(B)** `[1, 99, 3, 4, 5]` **(C)** `[99, 2, 3, 4, 5]` **(D)** `[1, 99, 99, 4, 5]`

Q7 | np.arange vs np.linspace

Which produces exactly 5 values from 0 to 1 (inclusive)?

(A) `np.arange(0, 1, 5)` **(B)** `np.linspace(0, 1, 5)` **(C)** `np.arange(0, 1, 0.25)` **(D)** `np.linspace(0, 1, 4)`

Q8 | Column Extraction

```
M = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(M[:, 1])
```

(A) `[1, 2, 3]` **(B)** `[2, 5, 8]` **(C)** `[4, 5, 6]` **(D)** `[3, 6, 9]`

2 1

Q9 | Boolean Mask Result

```
a = np.array([3, 7, 1, 9, 4, 6])
print(a[a > 5])
```

(A) `[7, 9, 6]` **(B)** `[3, 1, 4]` **(C)** `[7, 6]` **(D)** `[9, 6]`

Q10 | Compound Mask

```
a = np.array([2, 5, 8, 11, 14])
result = a[(a > 4) & (a < 12)]
print(result)
```

(A) `[5, 8, 11]` **(B)** `[5, 8]` **(C)** `[8, 11]` **(D)** `[5, 8, 11, 14]`

Q11 | np.where

```
a = np.array([1, -2, 3, -4, 5])
result = np.where(a > 0, a, 0)
print(result)
```

(A) `[1, -2, 3, -4, 5]` **(B)** `[1, 0, 3, 0, 5]` **(C)** `[0, -2, 0, -4, 0]` **(D)** `[1, 2, 3, 4, 5]`

Q12 | [HARD] Fancy Indexing

```
a = np.array([10, 20, 30, 40, 50])
idx = np.array([2, 0, 4])
print(a[idx])
```

(A) `[20, 10, 40]` **(B)** `[30, 10, 50]` **(C)** `[30, 20, 50]` **(D)** `Error`

Q13 | Vector Broadcast

```
a = np.array([[1],[2],[3]])    # shape (3,1)
b = np.array([10,20,30])       # shape (3,)
print((a + b).shape)
```

(A) `(3,)` **(B)** `(3,3)` **(C)** `(9,)` **(D)** `Error`

Q14 | Matrix Multiply

```
A = np.array([[1,2],[3,4]])
B = np.array([[1,0],[0,1]])    # identity
print(A @ B)
```

(A) `[[1,2],[3,4]]` **(B)** `[[1,0],[0,4]]` **(C)** `[[1,2],[0,4]]` **(D)** `[[2,2],[6,4]]`

Q15 | Axis Aggregation

```
a = np.array([[1,2,3],[4,5,6]])
print(np.sum(a, axis=0))
```

(A) `[5, 7, 9]` **(B)** `[6, 15]` **(C)** `21` **(D)** `[1,2,3,4,5,6]`

Q16 | [HARD] Stacking

```
a = np.array([1,2,3])
b = np.array([4,5,6])
print(np.vstack([a,b]).shape)
```

(A) `(6,)` **(B)** `(2,3)` **(C)** `(3,2)` **(D)** `(1,6)`

3 1

Q17 | read_csv Parameter

Which parameter in `pd.read_csv()` specifies that the **first column** should be used as the index?

(A) `header=0` **(B)** `index_col=0` **(C)** `usecols=0` **(D)** `skiprows=1`

Q18 | df.info()

What information does `df.info()` NOT directly provide?

(A) Non-null counts per column **(B)** Data type of each column **(C)** Mean of numeric columns **(D)** Memory usage

Q19 | dtypes

A column has `dtype = object` in Pandas. This most likely means:

(A) The column contains floats **(B)** The column contains strings or mixed types
(C) The column contains booleans **(D)** The column contains datetime objects

Q20 | Shape

```
df = pd.DataFrame({'A': [1,2,3], 'B': [4,5,6], 'C': [7,8,9]})
print(df.shape[0], df.shape[1])
```

(A) 3 3 **(B)** 9 3 **(C)** 3 9 **(D)** 3 2

4 1

Q21 | Null Detection

```
import pandas as pd, numpy as np
df = pd.DataFrame({'A':[1,np.nan,3], 'B':[np.nan,np.nan,6]})
print(df.isnull().sum().sum())
```

(A) 1 **(B)** 2 **(C)** 3 **(D)** 6

Q22 | dropna thresh

```
df = pd.DataFrame({
    'A':[1, np.nan, 3, np.nan],
    'B':[4, 5,      np.nan, np.nan],
    'C':[7, 8,      9,      np.nan]
})
print(df.dropna(thresh=2).shape)
```

(A) (4,3) **(B)** (3,3) **(C)** (2,3) **(D)** (1,3)

Q23 | fillna method

Which `fillna` method propagates the **last valid observation forward**?

(A) `method='bfill'` **(B)** `method='ffill'` **(C)** `method='mean'` **(D)** `method='zero'`

Q24 | Duplicate detection

```
df = pd.DataFrame({'Name':['A','B','A','C','B'],
                   'Age': [10, 20, 10, 30, 25]})
print(df.duplicated().sum())
```

(A) 0 **(B)** 1 **(C)** 2 **(D)** 3

Q25 | [HARD] drop_duplicates keep

```
df = pd.DataFrame({'X': [1,2,1,2,3]})
df2 = df.drop_duplicates(keep='last')
print(df2['X'].tolist())
```

(A) [1, 2, 3] (B) [1, 2, 1, 2, 3] (C) [2, 1, 3] (D) [1, 3]

Q26 | astype conversion

Which code correctly converts a column with string numbers to integers?

(A) df['col'].astype('str') (B) df['col'].astype(int)
(C) pd.to_numeric(df['col'], errors='ignore') (D) df['col'].convert()

Q27 | String cleaning

```
df = pd.DataFrame({'Name': ['  alice ', 'BOB', '  Charlie']})
df['Name'] = df['Name'].str.strip().str.title()
print(df['Name'].tolist())
```

(A) ['alice', 'BOB', 'Charlie'] (B) ['Alice', 'Bob', 'Charlie']
(C) ['ALICE', 'BOB', 'CHARLIE'] (D) ['alice', 'bob', 'charlie']

Q28 | [HARD] pd.to_numeric errors

```
s = pd.Series(['1', '2', 'three', '4'])
result = pd.to_numeric(s, errors='coerce')
print(result.isnull().sum())
```

(A) 0 (B) 1 (C) 2 (D) 4

5 1

Q29 | .loc vs .iloc

```
df = pd.DataFrame({'A':[10,20,30]}, index=[5,10,15])
print(df.iloc[1]['A'])
print(df.loc[10]['A'])
```

(A) 10 and 20 (B) 20 and 20 (C) 10 and 10 (D) 20 and 10

Q30 | [HARD] .loc slice inclusive

```
df = pd.DataFrame({'V': [1,2,3,4,5]}, index=['a','b','c','d',
    'e'])
print(df.loc['b':'d', 'V'].tolist())
```

(A) [2, 3] (B) [2, 3, 4] (C) [1, 2, 3, 4] (D) [2, 3, 4, 5]

Q31 | isin filter

```
df = pd.DataFrame({'Dept':['HR','IT','Finance','HR','IT']})
print(df[df['Dept'].isin(['HR','IT'])].shape[0])
```

(A) 2 (B) 3 (C) 4 (D) 5

Q32 | .query() syntax

Which `.query()` call correctly filters rows where Age > 25 AND Dept is `'IT'`?

(A) `df.query("Age > 25 & Dept = 'IT'")`
(B) `df.query("Age > 25 and Dept == 'IT'")`
(C) `df.query("Age > 25 AND Dept == IT")`
(D) `df.query(Age > 25 and Dept == 'IT')`

Q33 | between

```
s = pd.Series([1, 5, 10, 15, 20])
print(s[s.between(5, 15)].tolist())
```

(A) `[5, 10]` (B) `[5, 10, 15]` (C) `[1, 5, 10, 15]` (D) `[10]`

Q34 | [HARD] Chained filtering

```
df = pd.DataFrame({
    'A': [1,2,3,4,5],
    'B': ['x','y','x','y','x']
})
result = df[df['A'] > 2][df['B'] == 'x']
print(result.shape[0])
```

(A) 1 (B) 2 (C) 3 (D) Error | use & instead

Q35 | str.contains

```
df = pd.DataFrame({'City':['New York','New Delhi','Mumbai','
    Newark']})
print(df[df['City'].str.contains('New')].shape[0])
```

(A) 1 (B) 2 (C) 3 (D) 4

6 1

Q36 | groupby mean

```
df = pd.DataFrame({
    'Dept':   ['HR','IT','IT','HR'],
    'Salary': [50000,60000,70000,40000]
})
print(df.groupby('Dept')['Salary'].mean()['IT'])
```

(A) 60000 (B) 65000 (C) 70000 (D) 55000

Q37 | agg multiple

```
df = pd.DataFrame({
    'G': ['A','A','B','B'],
    'V': [10, 20, 30, 40]
})
res = df.groupby('G')['V'].agg(['min','max'])
print(res.loc['B','max'])
```

(A) 30 (B) 40 (C) 35 (D) 10

Q38 | transform

```
df = pd.DataFrame({
    'Dept':   ['HR','IT','IT','HR'],
    'Salary': [50000, 70000, 90000, 30000]
})
df['mean_sal'] = df.groupby('Dept')['Salary'].transform('mean
    ')
print(df['mean_sal'].tolist())
```

(A) [40000, 80000, 80000, 40000] (B) [80000, 40000, 40000, 80000]
(C) [40000, 40000, 80000, 80000] (D) [60000, 60000, 60000, 60000]

Q39 | [HARD] groupby filter

```
df = pd.DataFrame({
    'Dept':  ['HR','IT','IT','HR','Finance'],
    'Score': [80, 90, 85, 70, 95]
})
big = df.groupby('Dept').filter(lambda x: len(x) >= 2)
print(big.shape[0])
```

(A) 2 (B) 3 (C) 4 (D) 5

Q40 | Named aggregation

Which syntax correctly creates a column `avg` from the mean of `Score` in a named aggregation?

(A) `.agg(avg='mean('Score'))`
(B) `.agg(avg=('Score', 'mean'))`
(C) `avg': 'Score'.mean().agg(')`
(D) `.agg(avg=mean('Score'))`

Q41 | [HARD] groupby size vs count

What is the difference between `.groupby('X').size()` and `.groupby('X')['Y'].count()`?

(A) They always produce the same result
(B) `size()` includes NaN rows; `count()` excludes NaN in column Y
(C) `count()` includes NaN rows; `size()` excludes NaN
(D) `size()` returns a DataFrame; `count()` returns a scalar

7 1

Q42 | Inner join row count

```
df1 = pd.DataFrame({'id':[1,2,3], 'val':['a','b','c']})
df2 = pd.DataFrame({'id':[2,3,4], 'info':['x','y','z']})
result = pd.merge(df1, df2, on='id', how='inner')
print(result.shape[0])
```

(A) 1 **(B)** 2 **(C)** 3 **(D)** 4

Q43 | Left join NaN

```
df1 = pd.DataFrame({'id':[1,2,3],'val':['a','b','c']})
df2 = pd.DataFrame({'id':[2,3,4],'info':['x','y','z']})
result = pd.merge(df1, df2, on='id', how='left')
print(result['info'].isnull().sum())
```

(A) 0 **(B)** 1 **(C)** 2 **(D)** 3

Q44 | [HARD] Outer join shape

```
df1 = pd.DataFrame({'k':[1,2],    'v1':[10,20]})
df2 = pd.DataFrame({'k':[2,3,4], 'v2':[200,300,400]})
res = pd.merge(df1, df2, on='k', how='outer')
print(res.shape)
```

(A) (2,3) **(B)** (3,3) **(C)** (4,3) **(D)** (5,3)

Q45 | concat axis

```
df1 = pd.DataFrame({'A':[1,2],'B':[3,4]})
df2 = pd.DataFrame({'C':[5,6],'D':[7,8]})
res = pd.concat([df1, df2], axis=1)
print(res.shape)
```

(A) (4,2) **(B)** (2,4) **(C)** (4,4) **(D)** (2,2)

Q46 | merge suffixes

When merging two DataFrames that both have a column `val`, what happens to that column without specifying `suffixes`?

(A) An error is raised
(B) The column appears once (last one wins)
(C) Pandas auto-adds _x and _y suffixes
(D) The column is dropped entirely

Q47 | [HARD] merge on different keys

```
emp  = pd.DataFrame({'emp_id':[1,2], 'name':['A','B']})
dept = pd.DataFrame({'id':[1,2],     'dept':['HR','IT']})
res  = pd.merge(emp, dept, left_on='emp_id', right_on='id')
print(res.columns.tolist())
```

(A) ['emp_id','name','dept']
(B) ['emp_id','name','id','dept']
(C) ['id','name','dept']
(D) Error

8 1

Q48 | pivot_table aggfunc

In `pd.pivot_table()`, if there are **multiple values** for the same row-column combination and no `aggfunc` is specified, which function is used by default?

(A) sum **(B)** count **(C)** mean **(D)** first

Q49 | melt id_vars

```
df = pd.DataFrame({'Name':['Alice','Bob'],
                   'Math':[85,90], 'Sci':[78,88]})
long = pd.melt(df, id_vars=['Name'],
               var_name='Subject', value_name='Marks')
print(long.shape)
```

(A) (2,3) **(B)** (4,3) **(C)** (4,4) **(D)** (2,4)

Q50 | [HARD] pivot_table fill_value

What does the `fill_value` parameter do in `pd.pivot_table()`?

(A) Fills NaN in the original DataFrame before pivoting
(B) Replaces NaN values in the resulting pivot table
(C) Sets the default aggregation value to 0
(D) Fills missing index labels

Q51 | pivot vs pivot_table

What happens when you use `df.pivot()` but there are **duplicate** row-column combinations?

(A) It takes the mean automatically
(B) It raises a `ValueError`
(C) It silently keeps the last value
(D) It creates a MultiIndex

9 1

Q52 | describe output

```
df = pd.DataFrame({'X': [2, 4, 4, 4, 5, 5, 7, 9]})
print(round(df['X'].describe()['std'], 4))
```

(A) `1.8708` **(B)** `2.0` **(C)** `3.5` **(D)** `1.5`

Q53 | IQR calculation

For the dataset `[10, 20, 30, 40, 50, 60, 70, 80]`, the IQR is:

(A) `30` **(B)** `40` **(C)** `50` **(D)** `45`

Q54 | mode

```
s = pd.Series([1, 2, 2, 3, 3, 3, 4])
print(s.mode()[0])
```

(A) `2` **(B)** `3` **(C)** `2.5` **(D)** `1`

Q55 | [HARD] skewness interpretation

A dataset has `mean > median > mode`. Which statement is true?

(A) The distribution is left-skewed (negative skew)
(B) The distribution is right-skewed (positive skew)
(C) The distribution is symmetric
(D) The skewness cannot be determined without data

Q56 | quantile

```
s = pd.Series([10,20,30,40,50,60,70,80,90,100])
print(s.quantile(0.9))
```

(A) 90 **(B)** 91 **(C)** 90.5 **(D)** 95

Q57 | [HARD] std ddof

In Pandas, `df['col'].std()` computes standard deviation with `ddof=1` by default. What does `ddof=1` mean?

(A) Divides by N (population std)
(B) Divides by N-1 (sample std, Bessel's correction)
(C) Divides by N+1
(D) Divides by $\sqrt{N}$

Q58 | cumsum

```
s = pd.Series([1, 2, 3, 4, 5])
print(s.cumsum().tolist())
```

(A) `[1, 3, 5, 7, 9]` **(B)** `[1, 3, 6, 10, 15]` **(C)** `[1, 2, 6, 24, 120]**(D)** `[5, 4, 3, 2, 1]`

10 1

Q59 | value_counts normalize

```
s = pd.Series(['A','B','A','C','A','B'])
print(round(s.value_counts(normalize=True)['A'], 4))
```

(A) 0.3333 **(B)** 0.5 **(C)** 3 **(D)** 0.6667

Q60 | crosstab

```
df = pd.DataFrame({
    'Sex':  ['M','F','M','F','M'],
    'Pass': ['Y','Y','N','Y','Y']
})
ct = pd.crosstab(df['Sex'], df['Pass'])
print(ct.loc['M','Y'])
```

(A) 1 **(B)** 2 **(C)** 3 **(D)** 0

Q61 | [HARD] crosstab normalize

In `pd.crosstab(a, b, normalize='index')`, what do the values represent?

(A) Column proportions (each column sums to 1)
(B) Row proportions (each row sums to 1)
(C) Overall proportions (entire table sums to 1)
(D) Raw frequency counts

Q62 | nunique

```
df = pd.DataFrame({'City':['Delhi','Mumbai','Delhi','Chennai'
    ,'Mumbai','Delhi']})
print(df['City'].nunique())
```

(A) 2 **(B)** 3 **(C)** 6 **(D)** 4

11 1

Q63 | Pearson range

The Pearson correlation coefficient r always lies in which range?

(A) $[0, 1]$ **(B)** $[-1, 1]$ **(C)** $(-\infty, \infty)$ **(D)** $[0, \infty)$

Q64 | corr method

Which correlation method is most appropriate for **ordinal data** or data with outliers?

(A) Pearson **(B)** Spearman **(C)** Kendall **(D)** Point-biserial

Q65 | [HARD] Correlation interpretation

Two variables have $r = -0.85$. Which statement is correct?

(A) A weak negative relationship exists
(B) A strong negative linear relationship exists
(C) There is no relationship
(D) One variable causes a decrease in the other

Q66 | Correlation matrix diagonal

In a Pandas correlation matrix `df.corr()`, all diagonal values are:

(A) `0` **(B)** `0.5` **(C)** `1.0` **(D)** The variance of each column

Q67 | [HARD] Correlation vs Causation

```
df = pd.DataFrame({'X': [1,2,3,4,5],
                   'Y': [2,4,6,8,10]})
print(df['X'].corr(df['Y']))
```

(A) 0.5 **(B)** 0.0 **(C)** 1.0 **(D)** -1.0

Q68 | Heatmap color

In a seaborn heatmap of a correlation matrix with `cmap='coolwarm'`, which color represents a correlation close to **+1**?

(A) Blue (cool) **(B)** White (neutral) **(C)** Red (warm) **(D)** Green

12 1

Q69 | IQR fences

For a dataset with Q1 = 20, Q3 = 50, the **upper fence** using the 1.5 × IQR rule is:

(A) 65 **(B)** 95 **(C)** 50 **(D)** 80

Q70 | Z-score threshold

A data point is typically considered an outlier using the Z-score method when $|z|$ exceeds:

(A) 1 **(B)** 2 **(C)** 3 **(D)** 5

Q71 | [HARD] IQR code

```
import numpy as np, pandas as pd
s = pd.Series([10,12,11,13,100,14,12])
Q1, Q3 = s.quantile(0.25), s.quantile(0.75)
IQR = Q3 - Q1
outliers = s[(s < Q1 - 1.5*IQR) | (s > Q3 + 1.5*IQR)]
print(len(outliers))
```

(A) 0 **(B)** 1 **(C)** 2 **(D)** 3

Q72 | Box plot whisker

In a standard box plot, the upper whisker extends to:

(A) Q3 + 1.5 × IQR or the maximum value, whichever is **smaller**
(B) Q3 + 1.5 × IQR always
(C) The maximum value always
(D) Q3 + 3 × IQR

Q73 | [HARD] Z-score formula

The Z-score for a value x in a dataset with mean μ and standard deviation σ is:

(A) $z = \frac{x-\mu}{\sigma^2}$ **(B)** $z = \frac{\mu - x}{\sigma}$ **(C)** $z = \frac{x-\mu}{\sigma}$ **(D)** $z = \frac{x}{\sigma - \mu}$

Q74 | Outlier impact

Which measure of central tendency is LEAST affected by outliers?

(A) Mean **(B)** Mode **(C)** Median **(D)** Variance

13 1

Q75 | rolling mean NaN

```
s = pd.Series([1, 2, 3, 4, 5])
print(s.rolling(window=3).mean().tolist())
```

(A) `[nan, nan, 2.0, 3.0, 4.0]`
(B) `[1.0, 1.5, 2.0, 3.0, 4.0]`
(C) `[nan, 1.5, 2.0, 3.0, 4.0]`
(D) `[2.0, 3.0, 4.0, nan, nan]`

Q76 | expanding mean

```
s = pd.Series([10, 20, 30])
print(s.expanding().mean().tolist())
```

(A) `[10.0, 20.0, 30.0]` **(B)** `[10.0, 15.0, 20.0]` **(C)** `[nan, 15.0, 20.0]`
(D) `[20.0, 20.0, 20.0]`

Q77 | shift lag

```
s = pd.Series([5, 10, 15, 20])
print(s.shift(2).tolist())
```

(A) `[nan, nan, 5, 10]` **(B)** `[15, 20, nan, nan]` **(C)** `[nan, 5, 10, 15]`
(D) `[5, 10, nan, nan]`

Q78 | [HARD] pct_change

```
s = pd.Series([100, 150, 120, 180])
print(round(s.pct_change()[1], 4))
```

(A) `0.5` **(B)** `50.0` **(C)** `0.3333` **(D)** `1.5`

Q79 | rolling min_periods

```
s = pd.Series([4, 8, 15, 16, 23])
result = s.rolling(window=3, min_periods=2).mean()
print(round(result[1], 4))
```

(A) nan **(B)** 4.0 **(C)** 6.0 **(D)** 8.0

Q80 | [HARD] rank dense

```
s = pd.Series([30, 10, 20, 10, 30])
print(s.rank(method='dense', ascending=True).tolist())
```

(A) [3.0, 1.0, 2.0, 1.0, 3.0] **(B)** [4.5, 1.5, 3.0, 1.5, 4.5]
(C) [3.0, 1.0, 2.0, 2.0, 3.0] **(D)** [5.0, 1.0, 3.0, 2.0, 4.0]

Q81 | diff()

```
s = pd.Series([10, 15, 13, 20, 18])
print(s.diff(1).tolist())
```

(A) [nan, 5, -2, 7, -2] **(B)** [nan, 5, 2, 7, 2] **(C)** [5, -2, 7, -2, nan] **(D)** [0, 5, -2, 7, -2]

Q82 | [HARD] EWM span

In `s.ewm(span=3).mean()`, a higher **span** value results in:

(A) More weight on recent values (more reactive)
(B) Less weight on recent values (smoother, slower to react)
(C) Equal weight on all values
(D) No change — span has no effect on smoothing

14 1

Q83 | [HARD] apply lambda

```
df = pd.DataFrame({'Score': [50, 70, 85, 40, 95]})
df['Grade'] = df['Score'].apply(lambda x: 'A' if x>=80 else (
    'B' if x>=60 else 'C'))
print(df['Grade'].value_counts()['C'])
```

(A) 1 **(B)** 2 **(C)** 3 **(D)** 0

Q84 | assign

```
df = pd.DataFrame({'A':[1,2,3],'B':[4,5,6]})
df2 = df.assign(C = df['A'] + df['B'])
print(df2['C'].tolist())
```

(A) [5, 7, 9] (B) [1, 2, 3] (C) Error (D) [4, 5, 6]

Q85 | [HARD] map vs apply

```
df = pd.DataFrame({'Cat': ['A','B','A','C']})
mapping = {'A': 1, 'B': 2, 'C': 3}
df['Code'] = df['Cat'].map(mapping)
print(df['Code'].tolist())
```

(A) [1, 2, 1, 3] (B) [A, B, A, C] (C) [1, 2, 3, 4] (D) Error

Q86 | np.dot shape

```
a = np.array([[1,2,3]])    # shape (1,3)
b = np.array([[4],[5],[6]])  # shape (3,1)
print(np.dot(a, b).shape)
```

(A) (3,3) (B) (1,1) (C) (3,) (D) (1,3)

Q87 | sort_values

```
df = pd.DataFrame({'Name':['B','C','A'],'Score':[70,90,80]})
df2 = df.sort_values('Score', ascending=False)
print(df2['Name'].tolist())
```

(A) ['A','B','C'] (B) ['C','A','B'] (C) ['B','C','A'] (D) ['C','B','A']

Q88 | [HARD] MultiIndex groupby

```
df = pd.DataFrame({
    'Year': [2022,2022,2023,2023],
    'Q':    ['Q1','Q2','Q1','Q2'],
    'Rev':  [100, 150, 200, 250]
})
res = df.groupby(['Year','Q'])['Rev'].sum()
print(res[(2023,'Q2')])
```

(A) 200 (B) 250 (C) 450 (D) Error

Q89 | Boolean indexing result type

When you do `df[df['A'] > 3]`, the result is:

(A) A NumPy array (B) A Pandas Series (C) A Pandas DataFrame (D) A list

Q90 | [HARD] inplace gotcha

```
df = pd.DataFrame({'A': [1,2,3,None]})
df2 = df.dropna(inplace=True)
print(df2)
```

(A) A DataFrame with 3 rows
(B) `None`
(C) A DataFrame with 4 rows
(D) An error is raised

Q91 | [HARD] np.argmax 2D

```
a = np.array([[3,1,4],[1,5,9],[2,6,5]])
print(np.argmax(a))
```

(A) `5` **(B)** `8` **(C)** `(1,2)` **(D)** `9`

Q92 | set_index

```
df = pd.DataFrame({'ID':[10,20,30],'Val':['a','b','c']})
df.set_index('ID', inplace=True)
print(df.loc[20,'Val'])
```

(A) `'a'` **(B)** `'b'` **(C)** `'c'` **(D)** `KeyError`

Q93 | [HARD] groupby unstack

```
df = pd.DataFrame({'G':['A','A','B'],'C':['x','y','x'],'V'
    :[1,2,3]})
res = df.groupby(['G','C'])['V'].sum().unstack()
print(res.shape)
```

(A) `(2,1)` **(B)** `(2,2)` **(C)** `(3,2)` **(D)** `(1,2)`

Q94 | rolling corr

Which function computes a **rolling correlation** between two Pandas Series?

(A) `s1.rolling(n).corr(s2)` **(B)** `pd.rolling_corr(s1, s2, n)`
(C) `np.corrcoef(s1.rolling(n), s2)` **(D)** `s1.corr(s2, window=n)`

Q95 | [HARD] Axis confusion

```
df = pd.DataFrame({'A':[1,2],'B':[3,4],'C':[5,6]})
print(df.drop('B', axis=1).shape)
```

(A) `(2,3)` **(B)** `(1,3)` **(C)** `(2,2)` **(D)** `(3,2)`

Q96 | np.unique

```
a = np.array([3,1,4,1,5,9,2,6,5,3])
print(len(np.unique(a)))
```

(A) 6 **(B)** 7 **(C)** 8 **(D)** 10

Q97 | [HARD] chained groupby transform + filter

```
df = pd.DataFrame({
    'D': ['A','A','B','B','B'],
    'V': [10, 20, 5, 15, 25]
})
df['grp_mean'] = df.groupby('D')['V'].transform('mean')
result = df[df['V'] > df['grp_mean']]
print(result.shape[0])
```

(A) 1 **(B)** 2 **(C)** 3 **(D)** 4

Q98 | [HARD] Window rank within group

```
df = pd.DataFrame({
    'G':  ['X','X','Y','Y'],
    'Val':[30,10,40,20]
})
df['r'] = df.groupby('G')['Val'].rank(ascending=False, method
   ='dense')
print(df['r'].tolist())
```

(A) `[1.0, 2.0, 1.0, 2.0]` **(B)** `[2.0, 1.0, 2.0, 1.0]`
(C) `[1.0, 2.0, 2.0, 1.0]` **(D)** `[2.0, 1.0, 1.0, 2.0]`

Q99 | [HARD] Pivot then melt roundtrip

If you create a pivot table from a long-format DataFrame, then apply `melt()` with the same settings, you should get back:

(A) Exactly the same DataFrame (same order, no NaNs)
(B) The same data but potentially different row order and without aggregation artifacts
(C) An error because melt and pivot are not inverses
(D) A wider DataFrame with more columns

Q100 | [HARD] Memory & dtype optimization

You have a column with integer values between 0 and 100. Which dtype uses the LEAST memory?

(A) `int64` **(B)** `int32` **(C)** `int8` **(D)** `float32`

14 1

Answer Key

1. B	**26.** B	**51.** B	**76.** B
2. A	**27.** B	**52.** A	**77.** A
3. A	**28.** B	**53.** B	**78.** A
4. C	**29.** B	**54.** B	**79.** C
5. A	**30.** B	**55.** B	**80.** A
6. B	**31.** C	**56.** C	**81.** A
7. B	**32.** B	**57.** B	**82.** B
8. B	**33.** B	**58.** B	**83.** B
9. A	**34.** B	**59.** B	**84.** A
10. A	**35.** C	**60.** B	**85.** A
11. B	**36.** B	**61.** B	**86.** B
12. B	**37.** B	**62.** B	**87.** B
13. B	**38.** A	**63.** B	**88.** B
14. A	**39.** C	**64.** B	**89.** C
15. A	**40.** B	**65.** B	**90.** B
16. B	**41.** B	**66.** C	**91.** A
17. B	**42.** B	**67.** C	**92.** B
18. C	**43.** B	**68.** C	**93.** B
19. B	**44.** C	**69.** B	**94.** A
20. A	**45.** B	**70.** C	**95.** C
21. C	**46.** C	**71.** B	**96.** B
22. B	**47.** B	**72.** A	**97.** B
23. B	**48.** C	**73.** C	**98.** A
24. B	**49.** B	**74.** C	**99.** B
25. C	**50.** B	**75.** A	**100.** C

Score Interpretation

Score	Rating	Action
90–100	Excellent	Contest ready!
75–89	Good	Review weak sections
60–74	Average	Re-study notes + practise
Below 60	Needs work	Full syllabus revision